

LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: Vijayasri
Roll No.: A24126552262
Date: 12-09-2026

1. TITLE

Develop a Mini To-Do List Application Using HTML, CSS and JavaScript

2. AIM

To develop a simple interactive To-Do List web application using HTML, CSS and JavaScript that allows users to add tasks, mark tasks as completed, and delete tasks.

3. OBJECTIVE

- To create the structure of a To-Do List using HTML.
 - To style the application using CSS.
 - To select and manipulate HTML elements using the DOM.
 - To add tasks dynamically using JavaScript.
 - To validate empty task input.
 - To mark tasks as completed or active.
 - To delete tasks and update the empty-task message.
-

4. THEORY

The application uses HTML to define the task input area, Add Task button, empty-message paragraph and task list. CSS provides the layout, card appearance, buttons, task rows and completed-task styling. JavaScript uses the DOM to select elements and event listeners to respond to user actions.

When the Add Task button is clicked, the entered text is trimmed and checked. A non-empty value creates a new list item containing the task text, Complete button and Delete button. The Complete button toggles the **completed** class and changes its label. The Delete button removes the task item and displays the empty message when no tasks remain.

Basic flow: User enters task → JavaScript validates input → DOM creates task item → Complete/Delete events modify the task → interface is updated.

5. ALGORITHM / PROCEDURE

1. Create the HTML page containing the task input, Add Task button, empty message and task list.
2. Link the external CSS and JavaScript files.
3. Select the required HTML elements using `document.getElementById()`.
4. Attach a click event listener to the Add Task button.
5. Read and trim the task input value.
6. If the input is empty, display an alert and stop the operation.
7. Create a new list item and task-text element dynamically.
8. Create Complete and Delete buttons for the task.

9. Use the Complete button to toggle the completed class and button text.
 10. Use the Delete button to remove the task and show the empty message when the list becomes empty.
 11. Append the task to the list, hide the empty message, and clear the input field.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>

  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">

    <h1>My To-Do List</h1>

    <div class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task.....">
      <button id="addTaskBtn">Add Task</button>
    </div>
    <p id="emptyMessage">No tasks available.</p>

    <ul id="taskList"></ul>

  </div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
// Selecting HTML elements using DOM
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Add Task button event listener
addTaskBtn.addEventListener("click", function () {

  const taskText = taskInput.value.trim();
  // Check if input is empty
  if (taskText === "") {
    alert("Please enter a task.");
    return;
  }

  // Create a new list item
  const taskItem = document.createElement("li");
  taskItem.classList.add("task-item");

  // Create task text
  const taskSpan = document.createElement("span");
  taskSpan.textContent = taskText;
  taskSpan.classList.add("task-text");
  // Create buttons container
  const buttonsDiv = document.createElement("div");
  buttonsDiv.classList.add("task-buttons");

  // Create Complete button
  const completeBtn = document.createElement("button");
  completeBtn.textContent = "Complete";
  completeBtn.classList.add("complete-btn");

  // Create Delete button
  const deleteBtn = document.createElement("button");
  deleteBtn.textContent = "Delete";
  deleteBtn.classList.add("delete-btn");
  // Complete button event listener
  completeBtn.addEventListener("click", function () {
    taskSpan.classList.toggle("completed");

    if (taskSpan.classList.contains("completed")) {
      completeBtn.textContent = "Completed";
    } else {
      completeBtn.textContent = "Complete";
    }
  });

  // Delete button event listener
```

```

deleteBtn.addEventListener("click", function () {
    taskItem.remove();
    // Show empty message if no tasks remain
    if (taskList.children.length === 0) {
        emptyMessage.style.display = "block";
    }
});

// Add buttons to buttons container
buttonsDiv.appendChild(completeBtn);
buttonsDiv.appendChild(deleteBtn);

// Add task text and buttons to task item
taskItem.appendChild(taskSpan);
taskItem.appendChild(buttonsDiv);

// Add task item to task list
taskList.appendChild(taskItem);
// Hide empty message
emptyMessage.style.display = "none";

// Clear input box
taskInput.value = "";
});

```

style.css

```

body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 0;
}

.container {
    width: 500px;
    margin: 80px auto;
    background-color: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

h1 {
    text-align: center;
    margin-bottom: 25px;
}

.input-section {
    display: flex;
    gap: 10px;
}

#taskInput {
    flex: 1;
    padding: 10px;
    font-size: 16px;
}

#addTaskBtn {
    padding: 10px 15px;
    background-color: #4CAF50;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}

#addTaskBtn:hover {
    background-color: #45a049;
}

#emptyMessage {
    text-align: center;
    color: gray;
    margin-top: 25px;
}

#taskList {
    list-style: none;
    padding: 0;
    margin-top: 20px;
}

.task-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    background-color: #f8f8f8;
    padding: 12px;
    margin-bottom: 10px;
    border-radius: 5px;
}

.task-text {

```

```

        flex: 1;
        font-size: 16px;
    }

    .completed {
        text-decoration: line-through;
        color: gray;
    }

    .task-buttons {
        display: flex;
        gap: 5px;
    }

    .complete-btn {
        background-color: #2196F3;
        color: white;
        border: none;
        padding: 7px 10px;
        cursor: pointer;
        border-radius: 4px;
    }

    .delete-btn {
        background-color: #f44336;
        color: white;
        border: none;
        padding: 7px 10px;
        cursor: pointer;
        border-radius: 4px;
    }
}

```

7. CODE EXPLANATION

index.html – Defines the page structure, including the My To-Do List heading, task input, Add Task button, empty message, task list and links to CSS and JavaScript.

script.js – Uses DOM selection and click event listeners. It validates input, dynamically creates task elements, toggles completion state, deletes tasks, controls the empty message and clears the input.

style.css – Styles the page background and central container, input area, Add Task button, task rows, Complete/Delete buttons and the completed-task line-through effect.

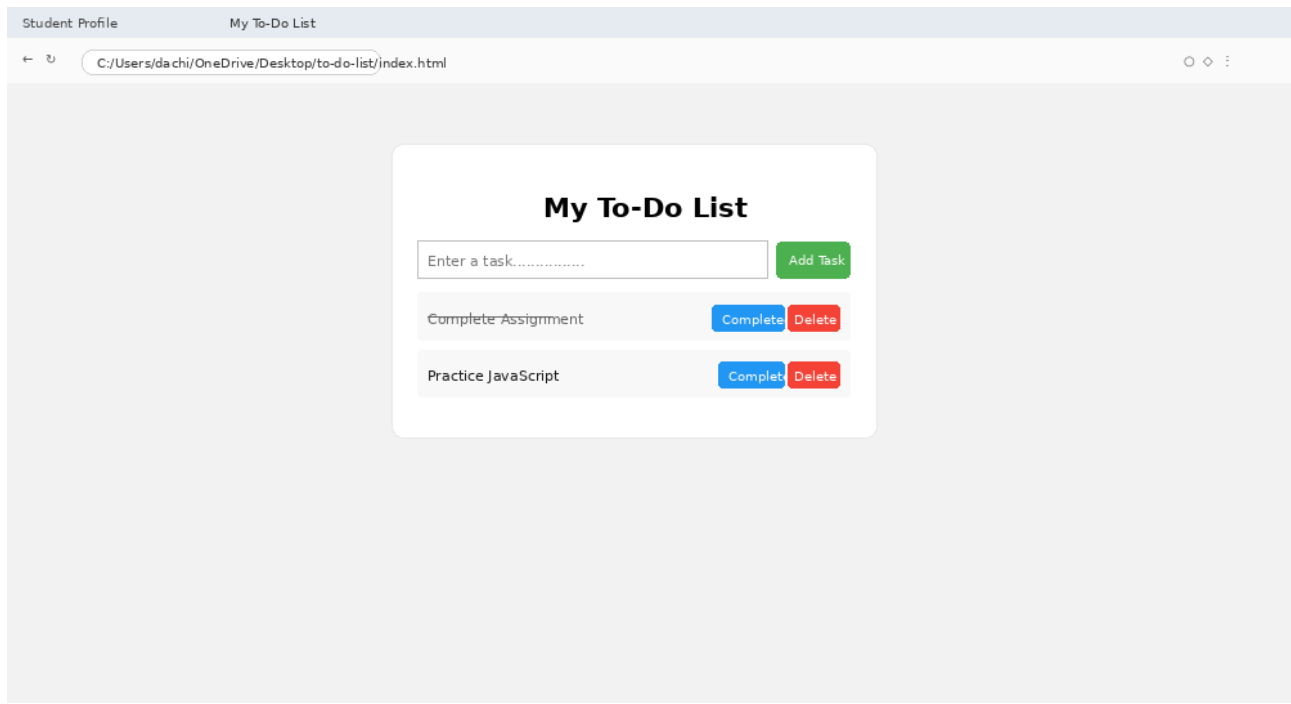
8. TEST CASES AND EXECUTION DATA

The following cases are based on the implemented event handlers and the supplied repository output screenshot. Where a behavior is not directly visible in the screenshot, it is identified as code-supported rather than claimed as an observed execution result.

Test Case	Input / Action	Expected Result	Actual / Evidence	Status
TC-01	Enter a task and click Add Task	Task should be added to the list.	Tasks are visible in the supplied output screenshot.	Pass
TC-02	Click Complete for a task	Task text should be crossed out and button should change to Completed.	Completed task with strikethrough and "Completed" button is visible in screenshot.	Pass
TC-03	Click Delete for a task	Task should be removed; empty message should appear when none remain.	Delete behavior is implemented in script.js; no deletion state is shown in the supplied screenshot.	Code verified
TC-04	Click Add Task with empty input	Alert "Please enter a task." should be displayed.	Empty-input validation is implemented in script.js; alert state is not shown in screenshot.	Code verified

9. OUTPUT

Output 1: To-Do List application showing added tasks, one completed task and active task.



10. RESULT

Thus, the Mini To-Do List application was developed using HTML, CSS and JavaScript. The supplied output demonstrates the task-list interface, task addition and completion state, while the program also implements task deletion and empty-input validation.